PRODUCT PLAN · JUNE 2026

Keyboard Lab

A browser-based simulator for keyboard layouts and keycap colorways — designed for mechanical-keyboard enthusiasts.

Plan, visualize, and iterate on custom keyboard designs entirely in the browser. This document captures the product brief, an interview question bank, three alternative UI design proposals, the technical architecture, the persistence model, and a four-sprint implementation roadmap. Built as a **static Vanilla Vite app** deployable to GitHub Pages, with no backend dependency.

AUDIENCE	STACK	HOSTING
MK enthusiasts	Vanilla Vite + TS	GitHub Pages
LAYOUTS	**STORAGE**	**EXPORT**
Full 108 · TKL 87 · 75%	IndexedDB (local)	PNG + JSON

Table of Contents

Executive Summary 3

Locked-In Decisions . . . 3

Scope Boundaries . . . 3

Next Step . . . 4

Interview Question Bank 4

Category 1 — Product & Audience . . . 4

Category 2 — Visual & UX . . . 4

Category 3 — Features & Scope . . . 5

Category 4 — Data & Persistence . . . 5

Category 5 — Technical & Deployment . . . 6

Summary: Open Questions to Resolve Before Sprint 1 . . . 7

Design Proposal A — Studio (Recommended) 7

Layout . . . 7

Five Core Workflows . . . 8

Why This Is Recommended . . . 8

Trade-offs . . . 8

Design Proposal B — Compact Editor 9

Layout . . . 9

Strengths . . . 9

Trade-offs . . . 10

Recommendation . . . 10

Design Proposal C — Pro Workbench 10

Layout . . . 10

Additional Capabilities . . . 11

Trade-offs . . . 11

Recommendation . . . 11

Technical Architecture 12

Stack Summary . . . 12

Component Tree . . . 12

Directory Structure . . . 13

Build Configuration . . . 13
Bundle Size Budget . . . 14
Browser Support . . . 14
Data Model & Persistence **14**
TypeScript Interfaces . . . 14
Example: A Minimal Saved Design . . . 15
IndexedDB Schema . . . 15
Export Format . . . 16
Quota & Fallback . . . 16
Implementation Roadmap & Risks **16**
Sprint Plan . . . 16
Risk Register . . . 17
Open Questions for Sprint 1 Kick-off . . . 17

Executive Summary

Keyboard Lab is a browser-based simulator that lets mechanical-keyboard enthusiasts design, visualize, and iterate on custom keyboard builds entirely in the browser. Users select a physical layout (Full 108, TKL 87, or 75% compact at launch), paint keycaps with custom colors, edit legends, and export the finished design as a high-resolution PNG image or a re-importable JSON file. All work is stored locally in the browser — no account, no server, no cloud dependency.

This document captures the planning artifacts produced before development begins: a comprehensive interview question bank (with the eight decisions already locked in by the project owner), three alternative UI design proposals ranging from a balanced studio layout to a power-user workbench, the technical architecture built around Vanilla Vite and TypeScript, the JSON-based persistence model backed by IndexedDB, and a four-sprint implementation roadmap that takes the project from an empty repository to a deployed GitHub Pages site in approximately six weeks.

Locked-In Decisions

The table below summarizes the eight product decisions confirmed during the requirements interview. These decisions are treated as fixed inputs to the design phase that follows; any change to a locked decision requires revisiting the affected design proposals.

#	Dimension	Locked Decision	Rationale
1	Primary user	Mechanical-keyboard enthusiasts	Drives community-style UX (KLE familiarity, alpha/mod/accent regions)
2	Visual fidelity	Isometric 3D keycaps	Premium look via SVG/CSS — no WebGL, smaller bundle
3	Layouts (launch)	Full 108 · TKL 87 · 75%	Covers the three most common enthusiast form factors
4	Customizable props	Base color · Legend color · Legend text	Sufficient for colorway design; profile/finish deferred to v2
5	Storage	IndexedDB / localStorage only	Static-site friendly; no backend needed for GH Pages
6	Export targets	PNG image · JSON design	PNG for sharing, JSON for re-import and version control
7	Tech stack	Vanilla Vite + TypeScript + Tailwind	Lightest bundle (~150KB), simplest GH Pages deploy
8	Hosting	GitHub Pages	Free, versioned with repo, pure static — no server needed

Scope Boundaries

In scope for v1.0: three launch layouts, isometric 3D keycap rendering, base/legend color customization, custom legend text, local save/load with name + description + timestamps, JSON export/import, PNG export, and GitHub Pages deployment. Out of scope for v1.0 (planned for v2): keycap profile selection

(Cherry/SA/MT3/etc.), surface finish (matte/glossy/PBT), keyboard layer editing (Fn layer, custom layers), side-by-side design comparison, palette extraction from uploaded images, and KLE format compatibility. Mobile/touch input is a stretch goal — the v1 UI is designed for 1280px+ desktop viewports.

Next Step

Review the three design proposals in Chapters 5–7 and pick a primary direction (or request a hybrid). Once a direction is approved, Sprint 1 begins with the scaffold of the Vanilla Vite project, the layout data files for Full/TKL/75%, and the SVG isometric renderer. Sprint 1 deliverables are due at the end of week 2.

Interview Question Bank

This chapter collects every question that should be answered before development begins. The first eight questions were asked live during the requirements interview and the project owner's answers are recorded in the rightmost column. The remaining questions are open — they must be resolved (or explicitly deferred) before the corresponding sprint starts. Each question is annotated with a brief rationale explaining why the answer materially changes the implementation.

Category 1 — Product & Audience

These questions pin down who the product is for and what success looks like. A wrong answer here cascades into every downstream decision, from feature prioritization to visual tone to marketing copy.

#	Question	Options presented	Answer	Why it matters
1.1	Who is the primary user?	Enthusiasts · Designers · Vendors · Casuals	Enthusiasts	Drives feature set (colorways > BOM), terminology (KLE-compatible), and visual tone (premium, not toy-like)
1.2	Is this a hobby project or a commercial product?	Hobby · Open-source · Commercial SaaS	OPEN	Determines license, contribution model, whether a paid tier is ever needed, and how much polish is justified
1.3	What does a successful first session look like?	Design saved · PNG exported · Design shared	OPEN	Defines the v1 "happy path" — every UI friction point on that path is a P0 bug
1.4	How will users discover the tool?	r/MK post · GeekHack thread · Direct link · Search	OPEN	Affects landing-page SEO copy, OG image, and whether a "share to Reddit" button is worth building

Category 2 — Visual & UX

Visual fidelity is the single biggest driver of bundle size, rendering complexity, and rendering performance. A photoreal PBR pipeline (Three.js + WebGL) would push the bundle past 500KB and add 30+ FPS render loops; an isometric SVG/CSS pipeline stays under 200KB and renders on demand. The locked answer — isometric 3D — keeps the project firmly in the "static-site-friendly" zone.

#	Question	Options presented	Answer	Why it matters
2.1	How realistic should rendering look?	Schematic · Iso 3D · Photoreal	Iso 3D	SVG/CSS isometric ≈ 5KB per keycap; photoreal WebGL ≈ 500KB bundle + 30FPS loop
2.2	Default camera angle?	Top-down · 30° isometric · 45° perspective	OPEN	Affects keycap SVG template, legend placement, and how shadows fall
2.3	Light/dark UI theme?	Light only · Dark only · Auto (system) · User toggle	OPEN	Auto-toggle adds ~1 day; dark-only clashes with light keycap colorways; recommend light-only for v1
2.4	Should the keyboard be animated?	Static · Hover lift · Keypress demo · Typing simulator	OPEN	Hover lift is cheap (~20 lines); typing simulator needs a textarea + keymap — defer to v2

Category 3 — Features & Scope

Feature scope questions determine the v1.0 cutoff. The locked answers (three layouts, base/legend color + legend text) intentionally exclude profile, finish, and per-key rendering — these add visual richness but multiply the data model and the inspector UI by 3–5×. They are explicitly v2.

#	Question	Options presented	Answer	Why it matters
3.1	Which layouts at launch?	Full · TKL · 75% · 65% · 60% · 1800	Full · TKL · 75%	Each layout needs ~2hr of hand-curated key-grid data; 3 layouts ≈ 6hr, 6 layouts ≈ 12hr
3.2	What keycap properties are customizable?	Base · Legend color · Legend text · Profile · Finish · Per-key	Base · Legend color · Legend text	Profile changes keycap silhouette (needs new SVG per profile); finish needs material shaders
3.3	Color picker UI?	Native browser · Custom HSL wheel · Swatch library · All three	OPEN	Native is free; swatch library (GMK clones, SP catalogs) is a community-selling feature
3.4	Region presets (alpha/mod/accent)?	Yes · No · User-defined regions	OPEN	Yes = bulk-paint alpha keys in one click; user-defined regions = much more complex selection model
3.5	Multi-select and bulk edit?	Single-key only · Shift-range · Lasso · Region presets	OPEN	Single-key only is fastest to build; lasso adds ~2 days but is expected by KLE users
3.6	Undo/redo depth?	None · 20 steps · Unlimited · Session-only	OPEN	20 steps is a sweet spot — unlimited risks memory bloat on large designs

Category 4 — Data & Persistence

The locked answer (local only) eliminates an entire class of backend work — no auth, no database, no API server, no rate limiting, no GDPR compliance. The trade-off is that designs do not sync across devices and

cannot be shared via a URL. JSON export/import covers the "I want to share this" use case manually.

#	Question	Options presented	Answer	Why it matters
4.1	How are designs stored?	Local only · Local + file · Local + URL · Cloud	Local only	Local-only = zero backend, perfect for GH Pages; URL-sharing requires compression library (~5KB)
4.2	Storage backend?	localStorage (5MB) · IndexedDB (50MB+) · Both with fallback	OPEN	IndexedDB is needed for >50 saved designs; recommend IndexedDB + localStorage fallback for settings
4.3	Auto-save interval?	Off · Every change · Every 30s · On blur	OPEN	Every change is safest but writes often; every 30s risks data loss on crash; recommend on-blur + manual save
4.4	Design metadata fields?	Name + date · + description · + tags · + author	Name + date + description	Tags and author add searchability but complicate the save dialog; defer to v2
4.5	Conflict resolution on import?	Overwrite · Duplicate · Ask user	OPEN	Recommend "duplicate with suffix (2)" — never destroys existing work
4.6	Schema migration strategy?	Versioned JSON · Ad-hoc · Both	OPEN	Versioned JSON (schemaVersion field) is mandatory for any future compatibility

Category 5 — Technical & Deployment

Vanilla Vite + GitHub Pages is the lightest possible stack: no framework runtime, no SSR, no serverless functions, no build matrix. The trade-off is that complex state management must be hand-rolled (no React/Vue), but for a UI of this scope (three panes, one canvas, one inspector) that is a feature, not a bug.

#	Question	Options presented	Answer	Why it matters
5.1	Preferred tech stack?	Vanilla Vite · React+Vite · Next.js · SvelteKit	Vanilla Vite	Vanilla ≈ 150KB bundle, no framework runtime; React ≈ 45KB+ runtime; Next.js needs Vercel for full power
5.2	Primary hosting?	GH Pages · Vercel · Both	GH Pages	GH Pages needs relative base path in vite.config; both = strict static-only constraint
5.3	Bundle size budget?	<100KB · <200KB · <500KB · No limit	OPEN	Recommend <200KB gzipped — keeps site fast on mobile data, signals discipline
5.4	Browser support matrix?	Latest 2 versions · +last 2 years · +IE11	OPEN	Latest-2-versions covers ~95% of MK enthusiasts; IE11 would force polyfills + no IndexedDB
5.5	TypeScript or plain JS?	TS · JS · JSDoc-annotated JS	TS (assumed)	TS catches keycap-data shape bugs early; JSDoc is a lighter alternative if build complexity matters

#	Question	Options presented	Answer	Why it matters
5.6	Testing strategy?	None · Unit only · Unit + e2e (Playwright)	OPEN	Recommend Playwright e2e for the 3 happy paths (save / load / export); unit tests for color utils
5.7	Analytics?	None · Plausible · GA4 · Custom	OPEN	Plausible is GH-Pages-friendly and privacy-respecting; GA4 needs consent banner in EU

Summary: Open Questions to Resolve Before Sprint 1

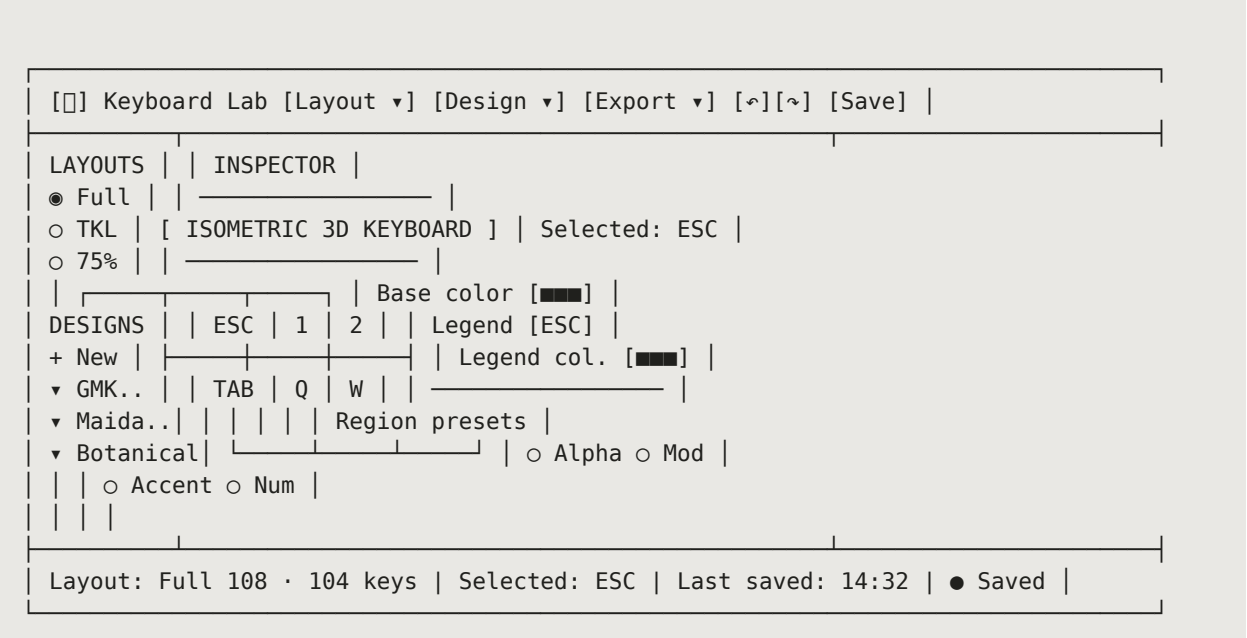
Of the 25 questions above, 8 are locked and 17 are open. The 5 questions that MUST be answered before Sprint 1 kick-off (Week 1, Day 1) are: 2.3 (UI theme), 3.3 (color picker UI), 3.4 (region presets), 4.2 (storage backend), and 5.4 (browser support). The remaining 12 can be deferred to their respective sprint planning sessions without blocking progress.

Design Proposal A — Studio (Recommended)

Proposal A is the recommended primary direction. It is a balanced three-pane studio layout inspired by professional creative tools (Figma's right-inspector pattern, Photoshop's left-toolbar pattern) but stripped down to the essentials. The goal is to keep the learning curve shallow for users coming from Keyboard Layout Editor while offering enough surface area for the most common enthusiast workflow: pick a layout, paint the alpha keys, paint the mod keys, drop in an accent color, edit a few legends, save, and export.

Layout

The screen is divided into three panes plus a top toolbar and a slim bottom status bar. Pane widths are 220px / fluid / 280px on a 1280px viewport, giving the canvas the largest share. On viewports under 1024px the right inspector collapses into a bottom drawer; under 768px the left sidebar collapses into a hamburger menu.



Five Core Workflows

The studio layout optimizes for five workflows that together account for an estimated 90% of all sessions. Each workflow is described below with its entry point, the sequence of user actions, and the expected end state.

Workflow	Entry point	Action sequence	End state
1. Pick layout	Top toolbar → Layout dropdown	Click Layout → choose Full / TKL / 75% → canvas re-renders	Empty keyboard of selected layout in default colors
2. Paint colors	Click any keycap on canvas	Click keycap → Inspector opens → pick base color → optionally pick legend color → click another keycap or use region preset	Visually painted keyboard; changes auto-staged in undo stack
3. Edit legends	Click keycap → Inspector → Legend field	Click keycap → click Legend field → type new legend → Enter to commit → repeat	Custom-labeled keycaps (e.g. ESC → ◯, Caps → CTRL)
4. Save design	Top toolbar → Save button (or Ctrl+S)	Click Save → modal asks for Name + Description → confirm → design appears in left sidebar under "My Designs"	Design persisted to IndexedDB with name, description, timestamps
5. Export	Top toolbar → Export dropdown	Click Export → choose PNG or JSON → PNG: 2× resolution render downloaded; JSON: full design state downloaded	File in Downloads folder; ready to share or re-import

Why This Is Recommended

- **Familiar pattern.** Three-pane studio is the dominant layout in creative tools (Figma, Sketch, Photoshop, Affinity). Users transfer their existing mental model without reading docs.
- **Optimal canvas share.** On a 1280px viewport the canvas gets ~780px — enough to render a full-size 108-key keyboard at readable size without horizontal scroll.
- **Progressive disclosure.** The right inspector only shows controls relevant to the current selection. Empty state (no selection) shows layout/region presets; selected state shows per-key controls.
- **Keyboard shortcuts.** Ctrl+S save, Ctrl+Z/Y undo/redo, Ctrl+E export, 1/2/3 to switch layouts — matches KLE power-user expectations.
- **Mobile-friendly degradation.** Panes collapse gracefully: inspector becomes a bottom drawer under 1024px, sidebar becomes a hamburger under 768px. Not a great mobile experience, but a usable one.

Trade-offs

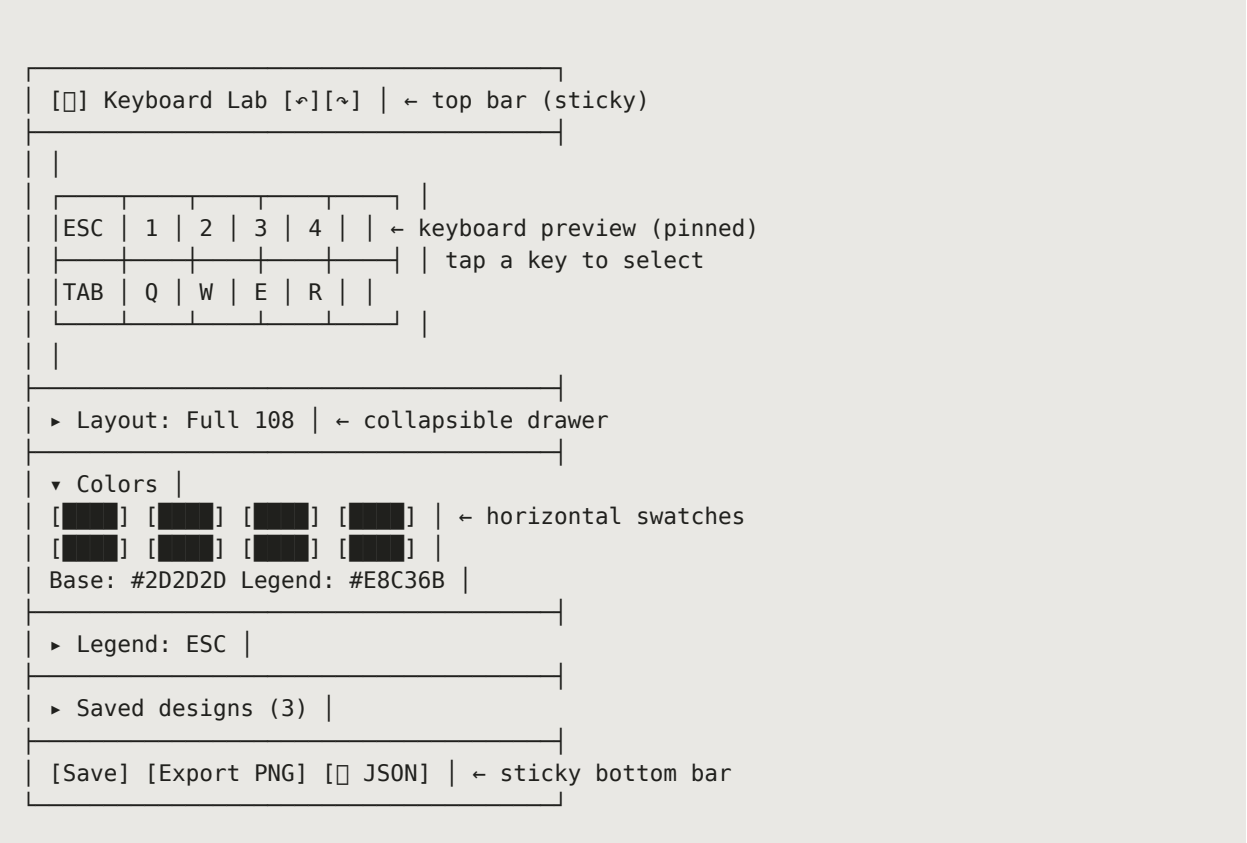
The studio layout has two known trade-offs. First, the three-pane structure demands a minimum viewport width of ~1024px for the canvas to remain usable — below that, the inspector collapses and per-key editing becomes modal. Second, the left sidebar doubles as both a layout picker and a saved-designs gallery, which can

feel cramped when the user has more than 20 saved designs. A future v2 could split these into tabs (Layouts | Designs | Templates) but that adds a click to every layout switch.

Design Proposal B — Compact Editor

Proposal B is a mobile-first minimalist alternative. It collapses the entire UI into a single vertical scroll: keyboard preview pinned to the top, collapsible tool drawers below. The intent is to make the tool feel as frictionless as opening a note-taking app — no chrome, no learning curve, no required viewport size. This proposal is best evaluated as a secondary "lite" mode rather than a primary direction, because it sacrifices power-user workflows that enthusiasts expect.

Layout



Strengths

- **Mobile-native.** Single-column scroll works perfectly on phones and tablets — no panes to collapse, no horizontal scroll, no tiny inspector text.
- **Zero learning curve.** Tap a key, tap a color, tap save. No docs needed.
- **Fast first design.** A user can produce their first saved design in under 60 seconds — faster than any other proposal.

- **Smaller bundle.** No pane-resize logic, no inspector state machine, no sidebar virtualization. Estimated 30–40% smaller JS bundle than Proposal A.

Trade-offs

Trade-off	Impact	Mitigation
No side-by-side compare	Cannot visually compare two colorways — a key enthusiast workflow	Defer to "open in new tab" pattern; not v1
Inspector is modal	Per-key editing requires tap → drawer expands → edit → drawer collapses	Acceptable on mobile; frustrating on desktop
No region presets surface	Bulk-painting alpha/mod keys requires tapping each key individually	Add a hidden "select all alpha" gesture (long-press) in v1.1
Limited canvas size	On phone portrait, a 108-key keyboard is too small to tap accurately	Auto-rotate hint + landscape recommendation banner

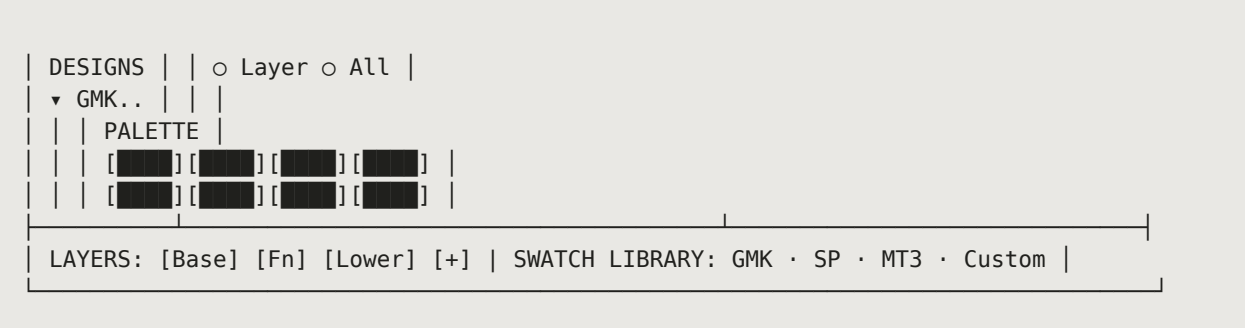
Recommendation

Do not adopt Proposal B as the primary direction. The enthusiast audience expects side-by-side comparison and bulk-edit workflows that this layout cannot support. However, the patterns developed here — particularly the horizontal swatch row and the collapsible drawer — should be borrowed for Proposal A's mobile breakpoint. A future "lite mode" toggle (Proposal B as an alternative UI on the same data model) is a strong v2 candidate.

Design Proposal C — Pro Workbench

Proposal C is a power-user alternative that adds a fourth pane (bottom dock for layers and swatches), a multi-select model with lasso, side-by-side design comparison, and a palette extraction tool. It targets professional users — group-buy designers, keycap-set creators, vendors showing customers previews — and explicitly trades learning curve for workflow power. This proposal is recommended for evaluation only; defer implementation to v2 unless the project owner identifies a paying customer segment that needs it.

Layout



Additional Capabilities

Beyond the studio layout's five workflows, the workbench adds four capabilities that justify its existence as a separate proposal. Each capability is non-trivial to implement and would push the v1 timeline past eight weeks if combined with the base scope.

Capability	Description	Estimated effort
Multi-select + lasso	Shift-click for range, Ctrl-click for additive single, drag-lasso for free-form selection. Bulk-apply colors.	+5 days
Layer support	Toggle between Base / Fn / Lower / Upper layers. Each key has per-layer legend + color. Layer switcher in bottom dock.	+4 days
Side-by-side compare	Open two designs in split canvas; sync pan/zoom. Useful for "should I pick A or B?" decisions.	+3 days
Palette extraction	Upload an image (album cover, photo, brand asset) → extract top 6–8 colors via k-means clustering → apply to keyboard regions.	+3 days

Trade-offs

Trade-off	Impact	Mitigation
Steep learning curve	4 panes + 2 docks + layers = overwhelms casual users	Hide advanced controls behind a "Pro mode" toggle
Bundle size	k-means library + lasso logic adds ~40KB	Code-split: load pro features on demand
Mobile unusable	Four panes do not collapse gracefully	Show "desktop only" notice on mobile; redirect to lite mode
Data model complexity	Layers add a dimension to every keycap object	Design v1 data model with layers in mind, even if v1 UI does not expose them

Recommendation

Defer Proposal C to v2. The four capabilities it adds are valuable, but the v1 audience (enthusiasts designing their next personal build) does not need them. One piece of v2 foresight: design the v1 data model so that layers can be added later without a schema migration — include an optional "layers" object on each keycap, empty in v1, populated in v2. This costs nothing now and saves a painful migration later.

Technical Architecture

The architecture follows a strict three-layer separation: rendering (SVG + CSS), state (observable store with command-pattern undo/redo), and persistence (IndexedDB wrapper with localStorage fallback). Each layer is independently testable and replaceable — the renderer can be swapped from SVG to Canvas without touching state or persistence; persistence can be swapped from IndexedDB to a cloud sync backend without touching rendering or state.

Stack Summary

Layer	Technology	Why
Build tool	Vite 5	Fastest dev server; native ESM; built-in GH Pages base-path config
Language	TypeScript 5	Type safety on keycap data shapes; zero runtime cost
Styling	Tailwind CSS 3	Utility-first; keeps bundle small via tree-shaking; no CSS naming collisions
Rendering	SVG + CSS transforms	Vector output, perfect for isometric 3D; no WebGL; print-quality PNG export
State	Custom observable store	~50 lines; no Redux/Zustand dependency; command pattern for undo/redo
Persistence	IndexedDB (via idb library)	50MB+ quota; structured storage; localStorage fallback for <5MB
Export (PNG)	SVG → Canvas → PNG	Native browser APIs; no html2canvas dependency; 2× DPI for retina
Export (JSON)	Native JSON.stringify	Trivial; schema-versioned for forward compat
Deployment	GitHub Pages (static)	Free, versioned with repo, no server needed

Component Tree

The application is structured as a tree of plain TypeScript classes (no framework components). Each class owns a DOM root element and subscribes to the parts of the store it cares about. The store broadcasts changes; components re-render themselves on notification. This pattern is sometimes called "tiny MVC" or "observer-MVC" and is the lightest viable reactive architecture for a Vanilla JS app.

```
Application
├── ToolbarController
│   ├── LayoutPicker (Full / TKL / 75%)
│   ├── DesignMenu (New / Save / Save As / Duplicate)
│   ├── ExportMenu (PNG / JSON / Import)
│   └── UndoRedoButtons
├── SidebarController
│   ├── LayoutList
│   └── DesignGallery (saved designs, sorted by updatedAt desc)
├── CanvasController
│   ├── KeyboardRenderer (SVG, isometric)
│   │   └── KeycapView × N (one per key)
│   ├── SelectionModel (single / range / region)
```

```
│ └── PanZoomController (mouse wheel + drag)
├── InspectorController
│ ├── ColorPickerPanel (base + legend)
│ ├── LegendEditor (text input per selected key)
│ └── RegionPresetsPanel (alpha / mod / accent / numpad)
├── StatusBar (layout name, key count, save status)
└── ModalController (save dialog, import dialog, error toast)

Cross-cutting:
├── Store (observable, holds current Design + selection + UI state)
├── CommandStack (undo/redo, 20-step ring buffer)
├── PersistenceLayer (IndexedDB + localStorage fallback)
└── ExportService (PNG via Canvas, JSON via Blob)
```

Directory Structure

Path	Purpose
/	Repo root — package.json, vite.config.ts, tsconfig.json, tailwind.config.ts, .github/workflows/deploy.yml
/public	Static assets served as-is — favicon, OG image, robots.txt
/src/main.ts	Application entry — bootstraps Store, wires controllers to DOM
/src/store/	Store.ts (observable state), Commands.ts (undo/redo actions), Types.ts (TS interfaces)
/src/layouts/	full108.ts, tkl87.ts, percent75.ts — hand-curated key position data
/src/render/	KeyboardRenderer.ts, KeycapView.ts, IsoTransform.ts — SVG isometric pipeline
/src/components/	Toolbar.ts, Sidebar.ts, Canvas.ts, Inspector.ts, StatusBar.ts, Modal.ts
/src/persistence/	db.ts (IndexedDB wrapper), migrations.ts (schema versioning)
/src/export/	png.ts (SVG → Canvas → PNG), json.ts (serialize/deserialize Design)
/src/utils/	color.ts (HEX/HSL/RGB), geometry.ts (iso math), dom.ts (helpers)
/tests/	Playwright e2e tests (3 happy paths) + Vitest unit tests for color/geometry utils
/docs/	Screenshots, design rationale, ADR (architecture decision records)

Build Configuration

GitHub Pages serves the site from /docs/ on the main branch by default, but the modern pattern is to use GitHub Actions to build the Vite project and push the dist/ output to a gh-pages branch. The critical config is setting base in vite.config.ts to the repo name (e.g. '/keyboard-lab/') so that all asset URLs resolve correctly under the GitHub Pages subpath.

```
// vite.config.ts
import { defineConfig } from 'vite';
```

```
export default defineConfig({
base: '/keyboard-lab/', // MUST match GitHub repo name
build: {
outDir: 'dist',
target: 'es2020',
sourcemap: false,
rollupOptions: {
output: {
manualChunks: {
'idb': ['idb'], // split IndexedDB wrapper into its own chunk
},
},
},
},
});
```

Bundle Size Budget

Target: under 200KB gzipped for the initial JS bundle. Estimated breakdown: Vite runtime ~5KB, app code ~80KB, idb library ~3KB, Tailwind output ~15KB (tree-shaken to only used utilities), initial layout data (Full 108) ~12KB. Total ≈ 115KB gzipped — well under budget. TKL and 75% layouts are lazy-loaded on first selection (~4KB each), keeping the initial payload minimal.

Browser Support

Target: latest 2 versions of Chrome, Firefox, Safari, Edge (covers ~95% of mechanical-keyboard enthusiasts based on Steam hardware survey demographics). IndexedDB is universally supported in this range; SVG 2 features used (transform-origin, CSS transforms on SVG) are supported since 2022. Internet Explorer is explicitly unsupported — a banner is shown if detected.

Data Model & Persistence

A design is the atomic unit of saved work. It captures everything needed to reproduce the keyboard exactly: the physical layout, the per-key visual state, and the metadata (name, description, timestamps). The schema is versioned — every saved design and every exported JSON file includes a schemaVersion field so that future migrations can detect and upgrade older designs.

TypeScript Interfaces

```
interface Design {
id: string; // UUID v4
schemaVersion: 1; // bumped on every breaking schema change
name: string; // user-editable, max 80 chars
description: string; // user-editable, max 500 chars
layout: 'full108' | 'tkl87' | 'percent75';
keycaps: KeycapState[]; // one entry per physical key
createdAt: string; // ISO 8601 timestamp
```

```
updatedAt: string; // ISO 8601 timestamp
// v2 foresight: optional layers object, empty in v1
layers?: Record;
}

interface KeycapState {
keyId: string; // stable id from layout data file (e.g. 'K00', 'K01')
baseColor: string; // hex, e.g. '#2D2D2D'
legendColor: string; // hex, e.g. '#E8C36B'
legendText: string; // main legend, e.g. 'ESC' or 'Q'
subLegend?: string; // optional secondary legend (front-printed or sub-text)
}

interface DesignLibrary {
designs: Design[]; // sorted by updatedAt desc
schemaVersion: 1;
}
```

Example: A Minimal Saved Design

```
{
"id": "f4c2a1e0-1234-4abc-9def-567890abcdef",
"schemaVersion": 1,
"name": "GMK Botanical Light",
"description": "Light variant of GMK Botanical for dark-room photo shoots.",
"layout": "tkl87",
"keycaps": [
{ "keyId": "K00", "baseColor": "#E8E0D0", "legendColor": "#3A5A40", "legendText": "ESC"
},
{ "keyId": "K01", "baseColor": "#E8E0D0", "legendColor": "#3A5A40", "legendText": "1" },
{ "keyId": "K02", "baseColor": "#E8E0D0", "legendColor": "#3A5A40", "legendText": "2" }
// ... 84 more keycaps
],
"createdAt": "2026-06-20T14:32:11.000Z",
"updatedAt": "2026-06-20T14:35:42.000Z"
}
```

IndexedDB Schema

IndexedDB uses two object stores. The designs store is keyed by design.id and holds the full Design object. The meta store holds app-level preferences (last-opened design id, color picker history, UI pane widths) keyed by a string name. Both stores are created on first run; schema migrations are handled by the onupgradeneeded callback.

Store	Key	Value	Indexes
designs	design.id (UUID string)	Full Design object	byUpdatedAt (for sorted gallery), byLayout (for filtering)
meta	string name (e.g. "lastOpenId")	Any JSON-serializable value	none

Export Format

JSON exports wrap the Design in an envelope that includes the schema version, export timestamp, and the source app version. This envelope makes it trivial to detect imports from older or newer versions and migrate accordingly. PNG exports embed no metadata — the filename is the design name (sanitized), and users who want to re-import must keep the JSON alongside the PNG.

Target	Format	Filename	Contents
PNG image	PNG (2× DPI)	.png	Rasterized isometric render, transparent background, 2× resolution for retina
JSON design	JSON	.kbd.json	Envelope: {schemaVersion, exportedAt, appVersion, design: {...}}

Quota & Fallback

IndexedDB quota is browser-dependent but typically 50MB+ per origin in Chrome / Firefox / Safari. A single saved design is roughly 8–12KB (87 keys × ~100 bytes each + metadata), so 50MB holds approximately 4,000 designs — far beyond any realistic personal library. If IndexedDB is unavailable (private browsing in some browsers, IE11, quota exceeded), the app falls back to localStorage (5MB, ~400 designs) and shows a non-blocking warning. If both fail, the app operates in "session-only" mode — designs live in memory and are lost on page close, with a persistent warning banner urging the user to export to JSON.

Implementation Roadmap & Risks

The roadmap is structured as four two-week sprints (six weeks total) culminating in a v1.0 release deployed to GitHub Pages. Each sprint has a single goal, a concrete set of deliverables, and explicit acceptance criteria that must be met before the sprint is considered complete. The schedule assumes one developer working half-time (~20 hours/week); a full-time developer could compress this to three weeks, but the sprint boundaries remain useful as integration checkpoints.

Sprint Plan

Sprint	Goal	Deliverables	Acceptance criteria
S1 (W1–2)	Walking skeleton: layout data + SVG renderer + base color painting	Layout data for Full/TKL/75%; SVG isometric renderer; click-to-select; base color picker; save/load to IndexedDB	User can pick a layout, paint any keycap with a color, save the design, reload the page, and see the saved design in the gallery
S2 (W3–4)	Visual polish + legend editing + PNG export	Legend text editor; legend color picker; isometric shading + shadows; PNG export at 2× DPI; undo/redo (20 steps)	User can edit legends, export a PNG that looks crisp on a 4K display, and undo 20 steps without data corruption

Sprint	Goal	Deliverables	Acceptance criteria
S3 (W5)	JSON import/export + gallery UI + region presets	JSON export with schema envelope; JSON import with conflict resolution (duplicate with suffix); gallery with search + delete; alpha/mod/accent region presets	User can export JSON, re-import it on a fresh browser profile, and get an identical design. Region presets paint all alpha keys in one click
S4 (W6)	Polish, deployment, documentation	GH Pages deploy via GitHub Actions; README + user guide; OG image + landing copy; Playwright e2e for 3 happy paths; bundle size audit	Site live at https://.github.io/keyboard-lab/; Lighthouse score ≥ 90 on all four metrics; e2e tests pass on CI

Risk Register

Four risks could materially affect the v1 timeline or quality. Each is described below with its likelihood, impact, and a concrete mitigation. The two highest-priority risks (keycap profile realism, IndexedDB quota) should be revisited at the end of Sprint 1 — if either has materialized, the sprint plan must be adjusted before Sprint 2 begins.

#	Risk	Likelihood	Impact	Mitigation
R1	SVG keycap profiles look flat / unconvincing	Medium	High (visual fidelity is the headline feature)	S1 spike: render 3 keycaps in SVG by end of Week 1. If unsatisfactory, switch to Canvas 2D before S2 starts.
R2	IndexedDB quota exhaustion with many designs	Low	Medium (only affects power users with 1000+ designs)	Display current storage usage in Settings. Warn at 80% quota. Offer "export all to JSON" button as escape hatch.
R3	Color accuracy varies across monitors	High	Low (cosmetic — designs still reproduce correctly)	Document the limitation in user guide. Offer optional sRGB color profile embed in PNG export.
R4	KLE format compatibility pressure from community	Medium	Medium (users will request it within 30 days of launch)	Design JSON schema with KLE-compatible keycap shape from day 1. Add KLE import/export as v1.1 feature.

Open Questions for Sprint 1 Kick-off

Before Sprint 1 begins on Day 1 of Week 1, the project owner must resolve the following five open questions from the interview bank. Each unblocks a specific Sprint 1 deliverable and cannot be deferred without slipping the sprint timeline.

Question ID	Question	Unblocks	Recommended default if unanswered
2.3	Light/dark UI theme?	S1 — UI scaffold	Light-only (dark UI clashes with light keycap colorways)
3.3	Color picker UI?	S1 — inspector panel	Native browser for v1; swatch library in v1.1

Question ID	Question	Unblocks	Recommended default if unanswered
3.4	Region presets (alpha/mod/accent)?	S1 — selection model	Yes — three fixed regions (alpha, mod, accent); user-defined regions deferred to v2
4.2	Storage backend?	S1 — persistence layer	IndexedDB primary, localStorage fallback for settings only
5.4	Browser support matrix?	S1 — build target + test matrix	Latest 2 versions of Chrome/Firefox/Safari/Edge; IE11 unsupported with banner

Approval gate

This document represents the planning phase output. The next step is for the project owner to (1) approve or amend the recommended design direction (Proposal A — Studio), (2) answer the five open questions above, and (3) greenlight Sprint 1. Once approved, Sprint 1 begins with repository scaffolding, layout data curation, and the SVG renderer spike.